Advanced Problem-Solving Lab – High-IQ Track

Activity H1 – Solve the "Hidden Problem" Behind the Stated Problem

Title: From Surface Problem to Real Problem

Goal
Train yourself to see the *real* problem, not just the stated one.

Instructions
You are given a "client statement":

"Our exam system is too slow. We want a Python script to speed it up."

1. Identify at least 3 possible *real* problems this statement might hide.
   - Hidden problem 1: Insufficient database queries
   - Hidden problem 2: poor algorithm design
   - Hidden problem 3: server overload (or) hard ware limitation
2. For each hidden problem, state:
   - What data would you need to confirm it?
     - Problem 1: Query execution time data box logs
     - Problem 2: code performance metrics, runtime analysis
     - Problem 3: cpu usage, Memory usage server logs.
3. Choose ONE hidden problem and re-frame it as a clear technical problem statement (2–3 lines):

New problem statement:
The system is slow due to insufficient database queries causing delay in fetching results. Optimize queries to reduce response time.

4. Design a very small Python experiment that could validate or reject your reframed problem (pseudo-code is fine):

python

 # Experiment idea (what will you measure, on what data, how?)
Measure time taken by current query v/s optimized query

```
start time = time.time()
run query()
end time = time.time()
print("Execution time", end time - start time)
```

Activity H2 – Multiple Competing Solutions

Title: Design 3 Solutions Before Coding 1

Goal
Avoid fixation. Explore at least three distinct solution strategies.

Instructions
Problem:

"You receive a stream of log lines (text). You must detect 'suspicious' activity in real time."

1. Give three different solution ideas, each using a distinct main technique or data structure.
   - Approach A (e.g., rule-based / regex / thresholds): Rule - based key word matching
   - Approach B (e.g., statistical / sliding window / counters): sliding window count of suspicious events
   - Approach C (e.g., simple ML / anomaly detection concept): Simple anomaly detection using ML
2. For each approach, list pros and cons (at least one each).
   - Approach A – Pros: ~~fast~~ simple/ Cons: limited accuracy
   - Approach B – Pros: ~~better~~ detection Cons: more computation
   - Approach C – Pros: ~~Adaptive~~ Cons: complex
3. Choose one approach to implement first and justify:

I choose approach ~~A~~ *because*
It is simple & easy to implement first
4. Sketch high-level Python structure (modules/classes, not full code):

python

 # Modules / classes / key functions list

~~#~~ Functions:
~~#~~ read_logs()
~~#~~ detect_suspicious()
~~#~~ alert_user()

Activity H3 – Self-Constrained Challenge: "Elegant Code Only"

Title: Solve with Elegance (Clarity, Minimalism, Correctness)

Goal

Produce code that is not only correct, but also minimal and beautiful.

Problem

Implement a mini "pipeline" for processing a list of tasks:

- Each task has an ID, description, and status ("todo", "doing", "done").
- You must:
  - Load tasks from a JSON file.
  - Filter by status.
  - Sort by ID.
  - Print a clean summary.

Constraints (you impose on yourself)

1. Maximum 25 lines of *real* Python (excluding blanks and comments).
2. No function longer than 7 lines.
3. Names must be meaningful and short.

Steps

1. Design your data model in 2–3 lines:

List of dictionaries with ID, description, Status

2. Decide at least one thing you will *not* do to keep it minimal:

Avoid complex classes to keep code simple

3. Write your code here (respecting constraints):

python

 # Elegant pipeline solution

4. Reflection:
   - What did you have to sacrifice (if anything) to keep elegance?

Reduced features to keep code short & clean

Code = Import

~~#~~ Load tasks

with open ("tasks.json") as :-

tasks = json.load (f)

~~#~~ filter + sort

tasks = [t for t in tasks if t ["status"] == "todo"]

tasks = sorted (tasks, key = lambda x : x["id"])

~~#~~ print summary

print t ["id"] :

Activity H4 – Complexity and Scaling Thought Experiment

Title: How Will Your Solution Break at Scale?

Goal

Develop realistic intuition about performance and scaling.

Instructions

Take ANY non-trivial function or algorithm you have recently written (search, summarization, scoring, etc.).

1. Describe what it does, in *one line*:
   Search for an element in a list
2. Estimate its time complexity in Big-O notation, and justify in 1–2 lines:
   - Big-O: O(n)
   - Because: It checks each element
3. Scaling scenarios – imagine input sizes:
   - $n = 10^3$ (1,000)
   - $n = 10^6$ (1,000,000)
   - $n = 10^9$ (1,000,000,000)

For each, predict: will it still be usable (Y/N) and why?

n	Usable (Y/N)	Reason (1 short sentence)
10^3	Yes	Fast
10^6	May be Yes	Slower
10^9	NO	TOO Slow

4. Suggest one change to improve scalability and briefly describe how it helps:
   We use set for faster look up !

Activity — Adversarial Thinking: Break Your Own Design

Title: Attack Your Own Program

Goal

Develop robustness and defensive mindset.

Instructions

Choose one of your own projects or mini-tools (file reader, API client, CLI, etc.).

1. Write a short description of what it is supposed to do:
   Reads file & processes data
2. Now wear the hat of an *attacker* or malicious user.
   List 5 ways to misuse / break it:
   - Attack 1: empty file
   - Attack 2: Invalid data
   - Attack 3: very large file
   - Attack 4: wrong file format
   - Attack 5: malicious input
3. For each attack, suggest one defence in plain language:
   - Defence for attack 1: Check file not empty
   - Defence for attack 2: validate input
   - Defence for attack 3: limit file size
   - Defence for attack 4: Check format
   - Defence for attack 5: sanitize input
4. Implement at least one of these defences in your code and paste the updated snippet:

python

 # Hardened code snippet:

```
if not data:
    print("File is empty")
try:
    value = int(data)
except:
    print("Invalid input")
```

Activity H6 – Cross-Domain Transfer

Title: Apply Known Pattern to a New Domain

Goal
Practice lifting a known idea/pattern from one domain to a different one.

Instructions
Pick a pattern you already know well, e.g.:

- Sliding window
- Two-pointer technique
- Dynamic programming table
- Producer–consumer
- Map–reduce style
- Pipeline of filters

1. Name of pattern: Sliding window
2. Original domain where you learned it (e.g., arrays, strings):
   Arrays
3. New domain to apply it in (e.g., logs, time series, learning activities, medical data):
   log data
4. Describe how you would adapt the pattern to this new domain, in 5–8 lines (can include pseudo-code):
   Use sliding window to track recent logs & detect frequent event in time window
5. Optional: implement a small prototype showing this transfer:

python

 # Pattern transfer prototype

```
window = logs[-5:]
for log in window:
    process(log)
```

Activity H7 – Meta-Cognition: Your Personal Problem-Solving Manual

Title: Write Your Own "How I Solve Problems" Guide

Goal

Turn your tacit skills into an explicit, reusable system.

Instructions

1. List the first three things you usually do when you get a new problem (be honest):
   - Step 1: Read problem
   - Step 2: Think solution idea
   - Step 3: Start coding
2. Now, design your ideal 6–8 step process (what a "future you" would always do):
   1. Understand the problem clearly
   2. Identify inputs & output
   3. Break problem into smaller steps
   4. Choose approach (logic / algorithm)
   5. Write code
   6. Test with example
   7. (optional) Fix errors (if any)
   8. (optional) Improve & optimize
3. For each step, write one question you will ask yourself:
   - Step 1 question: What is problem asking?
   - Step 2 question: What input & output needed?
   - Step 3 question: Can I divide the problem in steps?
   - Step 4 question: What is best way to solve it?
   - Step 5 question: Is my code correct?
   - Step 6 question: Does it work for all test cases.
4. Final instruction to yourself (one sentence):

"When I feel stuck, I will bread the problem into smaller steps one by one.

Activity I5 – Problem Template with Complexity Awareness

Title: Compare Two Approaches (Efficiency)

Problem

You must check if a number is present in a very large list (e.g., 1 million items).

Part A – Two Approaches

Approach 1: Simple loop search.

Approach 2: Use a set.

1. Describe Approach 1 in words:
   loop Checks each element
2. Describe Approach 2 in words:
   set allows faster look up

Part B – Implement Both

python

```
import time

# Create large list
nums = list(range(1000000))
target = 999999

# Approach 1: List search
start = time.time()
found = False
for x in nums:
    if x == target:
        found = True
        break
end = time.time()
print("List search found:", found, "Time:", end - start)

# Approach 2: Set search
num_set = set(nums)
start = time.time()
found2 = (target in num_set)
end = time.time()
print("Set search found:", found2, "Time:", end - start)
```

Reflection

- Which was faster and why (in simple words)?

Set is faster due to direct access

Activity I6 – Input Validation and Error Handling

Title: Robust User Input

Problem

You want a safe function that always returns a valid integer entered by the user.

Part A – Requirements

1. What should happen if user types "abc"?

   Show error for "abc"

2. What should happen if user leaves it blank?

   Asks again if blank present

Part B – Code with try/except

python

```
def get_int(prompt):
    while True:
        user_input = input(prompt)
        try:
            value = int(user_input)
            return value
        except ValueError:
            print("Invalid input. Please enter an integer.")

age = get_int("Enter your age: ")
print("Your age is:", age)
```

If 0 <= value <= 120:

else:

Print ("Please enter a value between 0 & 120".)

Tasks

- Modify get_int to also enforce a range (e.g., 0 to 120).
- If user enters out-of-range value, show a message and ask again.

If input is out of range (ex: 150) → ask again accepted only valid integer between 0 120

Activity I7 – Mini Project: Text Analyzer

Title: Word Frequency Counter

Problem

Given a short paragraph, count how many times each word appears.

Part A – Plan

Use the problem template (short version):

1. Inputs: paragraph
2. Outputs: word count
3. Steps (bullet points):
   - Convert to lowercase
   - Split words
   - Count frequency

Part B – Implement

python

```
text = input("Enter a sentence/paragraph: ")

# Convert to lower case
text = text.lower()


# Split into words
words = text.split()


# Count frequencies using dictionary
freq = {}
for w in words:
    if w in freq:
        freq[w] += 1
    else:
        freq[w] = 1


# Print results
for word, count in freq.items():
    print(word, ":", count)
```

Extensions (optional)

- Remove punctuation before counting.

- Show only words with frequency ≥ 2.

remove punctuation using strings module
print only words with frequency ≥ 2

* Remove punctuation before counting

```
import string
text = input ("Enter text:")
text = text.lower()
# Remove punctuation
for p in string.punctuation:
    text = text.replace(p, "")
words = text.split()
```

Activity I8 – Applying the Full Problem-Solving Workflow

Title: Generic Approach Practice – Loan EMI Calculator

Problem

Compute the Equated Monthly Instalment (EMI) for a loan given:

- Principal (P)
- Annual interest rate (R, in %)
- Tenure in years (N)

Formula (you may give to students):

$$EMI = \frac{P \cdot r \cdot (1+r)^n}{(1+r)^n - 1}$$

where

- $r = R/(12 \times 100)$ (monthly rate)
- $n = N \times 12$ (months)

Use Full Template

1. Problem understanding
   - Inputs: P.R.N
   - Outputs: EMI
   - Constraints: valid numbers
2. Analysis (steps)
   - convert rate
   - Calculate monthy
   - Apply formula
3. Design
   - Functions: e.g., compute_emi(P, R, N)? Yes / No
   - Data types: float/int? Both
4. Code

python

```
def compute_emi(P, R, N):
    r = R/(12*1000)
    n = N*12
    emi = (p * r * (1+r)**n) / ((1+r)**n-1)
    return emi

P = float(input("Principal: "))
R = float(input("Annual interest rate (%): "))
N = int(input("Tenure (years): "))
```

```
emi_value = compute_emi(P, R, N)
print("Monthly EMI =", emi_value)
```

5. Test

- Test 1: P = 100000 R = 10, N = 2 → EMI = 4614
- Test 2: P = 50,000, R = 8, N = 1 → EMI = 4340

Reflection

- Which step (understand/analyse/design/code/test) helped you most?

Design step helps most as it organises logic before coding.

Python Foundations – Activity Handouts

Activity 0.1 – Everyday Task as an Algorithm

Title: From Daily Routine to Algorithm

Instructions to Students

1. Choose one simple daily activity (examples: making tea, preparing to go to class, booking a cab).
2. Write the steps in clear, numbered points.
3. Make sure each step is:
   - Unambiguous
   - In correct order
   - Does not skip important details

Template

1. Activity chosen: Booking a Cab
2. Write the steps:
   1. Install the app.
   2. Login with your google or mobile no.
   3. Choose the destination
   4. Enter the Starting point.
   5. Select the Size of Vehicle.
   6. Pay the bill after your journey is Compl
3. Check your algorithm:
   - Can a friend follow it without asking questions? Yes / No
   - If "No", refine the steps here:
     -

Activity 0.2 – From Steps to Pseudo-code

Title: Writing Pseudo-code with IF and REPEAT

Instructions

1. Take your steps from Activity 0.1.
2. Identify where decisions happen (IF/ELSE).
3. Identify where repetition happens (REPEAT / WHILE).
4. Rewrite your algorithm using the following keywords:
   - IF, ELSE, ELSE IF
   - REPEAT, WHILE, UNTIL

Template

1. Original activity: Booking a Cab.
2. Pseudo-code version:

text

START

Size = Input("Enter the size")

IF Size = large. THEN

Print("You have to pay more")

ELSE

Print("Pay less")

ENDIF

REPEAT

UNTIL

END

3. Reflection:
   - Where did you use IF/ELSE? Size of the cab.
   - Where did you use REPEAT/WHILE?

Activity 1.1 – Run and Edit Given Code

Title: First Python Script – Edit and Run

Instructions

1. Your teacher will give you a small Python script (example below).
2. Do NOT write new code first; only change text or numbers.
3. Run the program and observe the output.

Example Script

python

```
name = "Student"
age = 18
print("Hello,", name)
print("You are", age, "years old.")
```

Tasks

1. Change name and age to your own values.
   - name: Sushanth.
   - age: 21
2. Run the program. What is the output?

text

Output:

Hello, Sushanth
You are, 21 Years old.

3. Change the printed message to:
   - "Welcome to Python, <your name>"

Write your edited code here: Print("Welcome to Python,", name)

python

```
# Edited code: Print("Welcome to Python,", name)
```

4. Reflection:
   - What changed in the output and why?

"Hello" Changed by "Welcome to Python" because ~~If~~ Commanded ~~to~~ Print so.

Activity 1.2 – Predict–Then–Run

Title: Predict Program Output

Instructions

1. Look at each code snippet.
2. BEFORE running, write what you think the output will be.
3. Then run it and compare.

Snippet A

python

```
x = 5
y = 2
z = x + y * 3
print(z)
```

- Predicted output: 11
- Actual output: 11

Snippet B

python

```
a = 10
b = a - 3
a = a + 1
print(a, b)
```

- Predicted output: (10, 7)
- Actual output: (11, 7)

Reflection

- Where was your prediction wrong? Why?

I used BODMAS rule so the value changed.

Activity 2.1 – Calculator in Steps

Title: From Word Problem to Code

Problem 1

A student has marks in 3 subjects. Find total and average.

1. Write the steps in plain language:
   1. add all the marks for Total.
   2. divide total by 3 to get average.
   3. _____
   4. _____
2. Now convert to Python (fill in the blanks):

python

mark1 = 70

mark2 = 80

mark3 = 90

total = 240

average = 80

```
print("Total =", total)
print("Average =", average)
```

3. Test with your own numbers and record output.

60, 70, 90, Total : 210, average = 70.

Activity 2.2 – Fix the Errors

Title: Debugging Simple Python

Instructions

Each snippet has an error.

1. Write what you think is wrong.
2. Correct the code.

Snippet 1

python

```
print("Hello World)
```

- Error: Double quoted coma is not completed.
- Corrected code: Print("Hello would")

python

undefined

Snippet 2

python

```
a = 10
b = "5"
c = a + b
print(c)
```

- Error: no need to use quoted comma for b=5
- Corrected code: b=5

python

undefined

Reflection:

- What did you learn about syntax vs logic errors?

Syntax errors will not execute whereas logical errors will execute with errors in results.

Activity 3.1 – Rule-based Classification

Title: Grades with IF/ELSE

Rule

- If mark ≥ 90 : Grade A
- Else if mark ≥ 75 : Grade B
- Else : Grade C

Tasks

1. Draw the decision in simple text:
   - Condition 1: if mark ≥ 90 then grade will be A.
   - Condition 2: if mark ≥ 75 then grade will be B.
2. Now write Python code:

python

```
mark = int(input("Enter mark: "))
if mark ≥ 90:
    print("Grade A")
elif mark ≥ 75:
    print("Grade B")
else:
    print("Grade C")
```

3. Test with at least 3 marks and note outputs.

---

mark = 80

Grade B

mark = 95

Grade A

mark = 70

Grade C

Activity 3.2 – Mini Problem Bank

Title: Structured Problem Solving with IF/ELSE

For each question, fill the template.

Template

1. Restate the problem in one sentence:

Pass or fail

2. Conditions needed (bullets):

- if marks >= 50 → Pass.
- marks < 50 → Fail.

3. Python code:

python

```
# Your solution
marks = int(input("Enter the marks"))
if marks >= 50:
    print("Pass")
else:
    print("Fail")
```

4. Test cases and results:

- Input: 60 Output: Pass
- Input: 49 Output: Fail

Use this template for:

- Odd or even number.
- Ticket discount if age < 12 or > 60.
- Check if a year is a leap year (simple rule).

- ```
  number = int(input("Enter the no:"))
  if number%2==0:
      print("Even number")
  else:
      print("odd number")
  ```

- ```
  age = int(input("Enter the age"))
  if age<12>age>60:
      print("Half ticket")
  else:
      print("Full ticket")
  ```

- ```
  Year = int(input("Enter the year"))
  if year%4==0:
      print("It is a leap year")
  else:
      print("Not leap year")
  ```
  ```

Activity 4.1 – Sum with Loops

Title: Sum from 1 to N

Steps

1. In plain language:
   - What is "sum from 1 to N"? $\frac{n}{2}(2+(n-1)) = \frac{n}{2}(1+n)$
2. Write the algorithm:

1. _____
2. _____
3. _____
4. _____

3. Now code it:

python

```
N = int(input("Enter N: "))
total = 0
for i in range(__N__):
    total = total + N(1+n)
print("Sum =", total)
```

4. Try N = 5, 10, 100 and check results.

Output

Enter N: 5

Sum = 15

Enter N: 10

Sum = 55

Enter N: 100

Sum = 5050.

Activity 4.2 – Star Patterns

Title: Loops for Patterns

Pattern (triangle)

```
*
**
***
****
```

1. Explain the pattern in words:

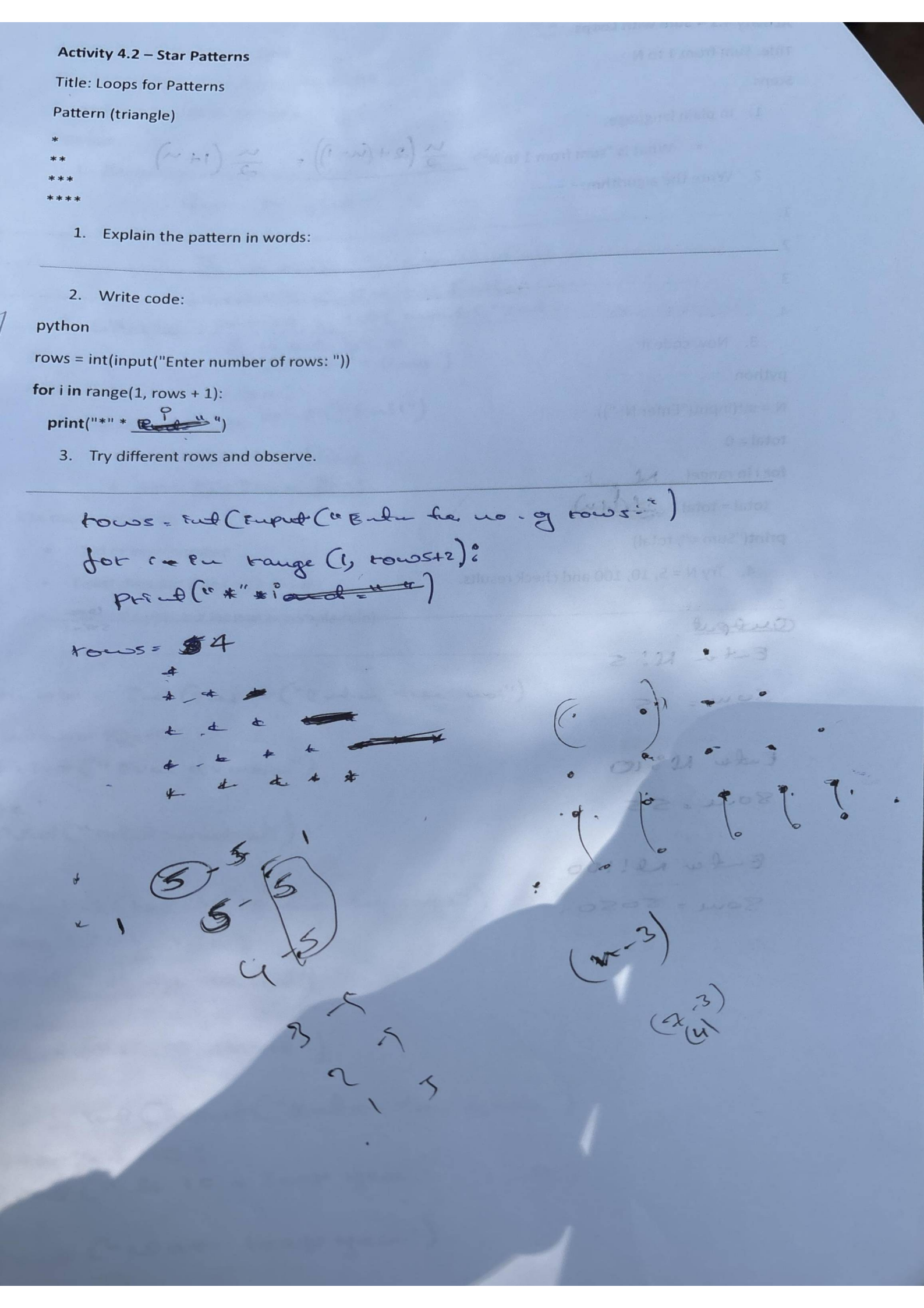
---

2. Write code:

python

```
rows = int(input("Enter number of rows: "))
for i in range(1, rows + 1):
    print("*" * i)
```

3. Try different rows and observe.



Activity 5.1 – Working with Lists

Title: Analyze Marks List

Given

python

```
marks = [56, 78, 90, 45, 88]
```

1. In words, write how to find:
   - Maximum mark: max(marks)
   - Average mark: sum(marks) / len(marks)
   - Count of marks ≥ 60: _____
2. Now write Python:

python

```
marks = [56, 78, 90, 45, 88]


# Max
max_mark = max(marks)


# Average
total = 0
for m in marks:
    total = total + m
average = total/len(marks)


# Count >= 60
count = 0
for m in marks:
    if m>=60:
        count = count + 1


print("Max:", max_mark)
print("Average:", average)
print("Count >= 60:", count)
```

Activity 5.2 – Choosing an Approach

Title: Which Method is Better?

Problem
Given a list of 1000 numbers, find how many are positive.

1. Approach A (loop) – describe in words:

   Running the iteration through entire no.

2. Approach B (using built-in functions or list comprehension) – describe:

   using sum() or len() function.

3. Which is easier to understand for you now? Why?

   looping is easier

4. Implement one approach in Python:

python

numbers = [10, -3, 5, 0, 8, -2] *# example*

 # Your code

Code:

```
count = 0
for i in numbers:
    if i > 0:
        count = count + 1
print("No. of +ve integers is:", count)
```

Activity 6.1 – Creating a Function

Title: Wrap Logic into a Function

Task

Turn average-of-marks logic into a function.

1. In words: what does the function do?

   To get the average

2. Write code:

python

```
def average_marks(marks_list):
    total = 0
    for m in marks_list:
        total = total + m
    avg = total/len(marks)
    return avg

data = [60, 70, 80]
print("Average =", average_marks(data))
```

3. Test with another list and note output.

Data = [70, 80, 90]

output = ~~80~~

Average = 80.

Activity 6.2 – Problem-Solving Template

Title: Standard Template for Any Problem

Use this for any new problem.

1. Problem understanding
   - What is given (inputs)?

     Pests, ~~Damage %~~, ~~Humidity~~.
   - What is required (outputs)?

     Yield loss ~~Calculation~~ Estimation.
   - Any constraints (limits, ranges)? ~~50%~~, ~~25%~~.

     Pest level = 30%, ~~Damage 25%~~, ~~Humidity 78%~~
2. Analysis
   - Write steps in bullets (algorithm):
     - Set critical value
     - ~~Analyse the~~ ~~procedure~~ of Pest, ~~Damage~~, ~~Humidity~~
     - Enter the Procedure of Pest, ~~Damage~~, ~~Humidity~~
     - Analyze the yield based on Procedure value.
3. Design
   - Will you use loops? Which? No
   - Will you use conditions (if/else)? Yes
   - Will you use functions? Which ones? No
   - What data structures (list, etc.)? No
4. Code (Python)

python

```
# Your implementation
Pest = int(input("Enter the no. of Pests"))
Print(f"Pests: {Pest}%")
if Pest > 50 :
  Print("Yield reduce by 75%.")
elif Pest > 30:
  Print("Yield reduce by 50%.")
elif Pest > 10:
  Print("Yield reduce by 20%.")
else:
  Print("No yield loss")
```

5. Test
   - Test case 1: input 35 → output Yield reduce by 50%.
   - Test case 2: input 55 → output Yield reduce by 75%.
   - Edge case: input 15 → output Yield reduce by 20%.

Activity 7.1 – Capstone Problem

Title: Simple Fee Calculator

Problem

A class charges a base fee.

- If student is below 12 years, 50% discount.
- If above 60, 30% discount.
- Otherwise, no discount.

Use the full template from Activity 6.2 here.

(Attach a blank copy of the template and ask students to fill all five sections.)

```
age = int(input("Enter the age"))
print(f"age:{age}")
if age > 60:
    print("you get 30% discount")
elif age < 12:
    print("you get 50% discount")
else:
    print("Pay full fee")
```

Problem understaing

Input: age, output: fee Payment

analysis

Set a particular value for which discount should be applied.

design:

elif condition

Code :- Python

Test

Input: 9 ; output: you get 50% discount

Activity 7.2 – Reflection Sheet

Title: How I Solve a New Problem

1. When I see a new problem, my first step is:

   analyze the situation.

2. Before coding, I make sure I know:

   - Inputs: The value which needs to be considered for output
   - Outputs: The result from applying different inputs.

3. The tools I think about (loops, if, functions) are:

   if else, elif, for, while.

4. One thing I will do differently from now on:

   learn all the tools for what it is used and use it in proper pattern to get the accurate outcome.

Python – Advanced Activity Handouts (Project & Problem-Solving Focus)

Activity A1 – From Idea to Project Specification

Title: Define Your Mini-Project Clearly

Goal

Turn a rough idea into a precise, buildable project description.

Instructions

Pick one idea (examples: simple hospital appointment system, dataset analyzer, quiz app, log analyzer, file organizer).

Part A – One-page project brief

1. Project title: Employment Generation application

2. Problem you want to solve (2–3 short sentences):
   To solve employment problem both for employer and ~~owner~~ employee

3. Target users (who will actually use this?):
   ~~Owner of work~~ of employer & labour.

4. Inputs (what data / actions will the program receive?):
   - Skills of labour, Price/salary of labour.
   - Region of labour, work to be done.

5. Outputs (what will the user see / get?):
   - opportunities of work for labour.
   - availability of labours in the region who are willing to work.

6. Constraints or assumptions (keep it realistic):
   - The labour may not use the app
   - because of illiteration.

Part B – Core features vs "nice-to-have"

1. List minimum features needed for a working version (MVP):
   - Labour can create a profile, search for local jobs
   - Employer can create a profile and send a
   - Call letter for labours.

2. List optional features (only if time allows):
   - Offline mode
   - filters for wages, skills.

3. Final question:
   - If you had only 1 week, which ONE feature would you absolutely keep? Why?

Offline mode, because most of the labours live in remote areas and are unavailable of Internet.

Activity A2 – System Decomposition (Modules & Responsibilities)

Title: Break Project into Modules

Goal
Learn to split a project into logical modules and functions.

Instructions
Use your project from Activity A1.

Part A – High-level modules

1. List 3–6 modules (e.g., data_input, processing, reporting, ui):
   - Module 1: Data Input Processing
   - Module 2: Processing
   - Module 3: User Interface
   - Module 4: Notification
2. For each module, write its main responsibility in one sentence:
   - Module 1 does: Capture data of users & job data.
   - Module 2 does: Matching labour to relevant jobs.
   - Module 3 does: Interaction btw labour & employer.
   - Module 4 does: To notify users.

Part B – Functions inside modules

For one key module (choose the most important):

1. Module name: data_input.
2. Functions planned:
   - Function 1 name: Create user
     - Input(s): user id, name, role
     - Output: User object.
   - Function 2 name: add skill
     - Input(s): user object, skill_list.
     - Output: updated skills in user object.
   - Function 3 name: update location
     - Input(s): user object, new location
     - Output: updated location in user object.
3. Draw or briefly describe how data flows between these functions:

The user will create ws user id and add his skill and update ws location

Designing Data Structures

Title: Choosing the Right Data Structures

Goal

Practice selecting lists, dicts, sets, tuples, or custom classes.

Instructions

Consider your project's core data (e.g., patient records, quiz questions, log entries, transactions).

Part A – Describe one key entity

1. Entity name (e.g., "Appointment", "Record", "Transaction"):
   User records
2. For this entity, list required fields:
   - Field 1: name (type: int/str/float/...)
   - Field 2: age (type: int)
   - Field 3: skill (type: str)
   - Field 4: exp (type: float)

Part B – Pick implementation

Choose one representation and justify:

1. Option A – Dictionary:

```
python
item = {
  "field1": _____,
  "field2": _____,
  ...
}
```

2. Option B – Class:

```
python
class Item:
  def __init__(self, field1, field2, ...):
    self.field1 = field1
    self.field2 = field2
    ...
```

3. Which option will you use, and why?

_____

4. Write a short example of creating and using this structure:

```
python
# Example creation and simple usage
```

_____

Activity A4 – Planning Tests Before Coding

Title: Think in Test Cases (TDD Lite)

Goal
Build habit of designing tests before writing code.

Instructions
Choose one non-trivial function from your project (e.g., fee calculation, filter logic, scoring rule).

1. Function name: Salary
2. In simple words, what should this function do?
   Fix the value for salary based on freelancer work.
3. List at least 5 test cases (inputs and expected outputs):

Test #	Input(s)	Expected output
1 Registration	name, age	displays the user credentials.
2 Region	Set the location.	The location of the user will be detected
3 Salary input.	Select the Salary range.	The lawyers with selected range areas.
4 Notification	Send a job request.	notify a employer device.
5 ~~Selection~~ of filters	apply any jithes with age /skill	Results for display of applied jithes.

4. Now write a small test harness:

python

```
def my_func(...):
    # TODO: implement
    pass


# Tests
print("Test 1:", my_func(...), "expected:", ...)
print("Test 2:", my_func(...), "expected:", ...)
```

5. Reflection:
   - Did writing tests first change how you thought about the function? How?

Error Handling & Robustness in a Project

Title: Make Your Project Hard to Break

Goal

Identify and protect weak points with validation and exception handling.

Instructions

Look at your project idea and identify three likely failure points (e.g., wrong file path, invalid input, network issue).

1. Failure Point 1: Network issue.
   - How to detect it? Check if the browser is connected to Internet.
   - How to handle it politely for the user? Provide guide for system
2. Failure Point 2: lagging/stuck.
   - Detect: lagging in the screen.
   - Handle: Suppose update the software.
3. Failure Point 3: _____
   - Detect: _____
   - Handle: _____

Part B – Add try/except around one risky area

Example skeleton:

python

```
file_path = input("Enter file path: ")

try:
    with open(file_path, "r") as f:
        data = f.read()
    print("File loaded successfully.")
except FileNotFoundError:
    print("File not found. Please check the path.")
except PermissionError:
    print("Permission denied. Try a different file.")
```

Adapt this pattern to your project and paste your modified code below:

python

 # Your robust code:

```
def calculate yield (rainfall, temperature, soil
    base - yield )
# rainfall
    if rainfall -> 600
```

Activity A6 – Refactoring for Clarity

Title: Improve Existing Code (Same Behaviour, Better Design)

Goal

Practice cleaning up messy, working code into clear, modular code.

Instructions

Take one working script (from your own work or given by teacher).
It should be correct but long, repetitive, or hard to read.

1. Paste or summarize original code name / purpose:

crop yield prediction program Calculate rainfall, temp, Soil quality

2. Identify at least three problems (long function, repetition, confusing names, etc.):

   - Problem 1: long function handles multiple responsibilities
   - Problem 2: Repeated calculation patterns for increasing/decreasing
   - Problem 3: Hard code numbers without explanation yield

3. Plan refactoring steps (high level):

   - Step 1: Separate rainfall temp Soil quality calculation into small func
   - Step 2: Replace no with named constant
   - Step 3: Improve readability with variable names.

4. After refactoring, write or paste your improved version here:

python

 # Refactored code:

5. Reflection:

   - What became easier to understand after refactoring?

After refractoring the code become easier to understand each function now handled only one task.

...g a Command-Line Interface (CLI)

...e: Make Your Project Usable from Terminal

Goal

Learn to take arguments and options from the command line.

Instructions

Turn some functionality of your project into a small CLI tool.

1. Define a simple CLI command:
   Example:
   - python student_tool.py add --name Alice --score 85
2. What should the tool do in ONE sentence?
   This tool allows users to input student names & display them through command line
3. Sketch CLI arguments:

Argument / Option	Meaning
Name	Name of student
Score	Score of student

4. Implement with argparse (skeleton):

```
python

import argparse


parser = argparse.ArgumentParser(description="Describe your tool here")
parser.add_argument("--name", type=str, help="Student name")
parser.add_argument("--score", type=int, help="Student score")
args = parser.parse_args()


# Use args.name, args.score here
print("Name:", args.name)
print("Score:", args.score)
```

5. Adapt the skeleton to your project and note example commands at the bottom of your file.

quiz.tool.py - name yash score 85

quiz.tool.py - name hush. score 90.

Activity A8 – Mini Capstone: End-to-End Project Plan & Review

Title: From Zero to Demo

Goal
Produce a realistic plan and self-review for your project.

Part A – Milestones and Timeline

1. List 4–6 milestones (each should be testable):
   - Milestone 1: Design question data structure
   - Milestone 2: load questions from file
   - Milestone 3: Implement quiz flow task question
   - Milestone 4: calculate & display score
   - Milestone 5: Add basic CLI support

2. For each milestone, define:

Milestone	Concrete Deliverable	Due Date
1	Question format ready	Day 1
2	Questions successfully	Day 2
3	Quiz runs with user input	Day 3

Part B – Risk & Mitigation

1. Identify 2–3 biggest risks (technical or time):
   - Risk 1: problem fails to read files
     - Plan B if this fails: embed coded questions
   - Risk 2: Not enough time to finish features
     - Plan B if this fails: focus only one core quiz functionaly
   - Risk 3: logical bugs in scoring
     - Plan B if this fails: Add test cases & debug step by step

Part C – Final reflection after project

(Do this at the end.)

1. When I got stuck, what strategies actually helped me get unstuck?
Breaking problem into smaller using print statement for debugging

2. One thing I now believe about myself as a problem-solver:
I can solve complex problem by approaching them step by st

3. If I start a new project tomorrow, what will I do differently from day 1?
I plan the structure, write test cases early & divide the project into clear no dues before coding